UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Department of Computer Science

COS 132 - Imperative Programming
Study Unit 7: Managing Memory (Prelim version 0.1)

Contents

7.1 Introduction

In this study unit we will revisit functions and specifically passing parameters by reference and recursion. The rest of the study unit will focus on memory and the management thereof. It will consider stack and heap memory and how values of types are stored on the stack and the heap and how these values are accessed. The study unit will conclude with a discussion of accessing string objects defined in the `string` library with pointers.

7.2 Functions reprised

C++ programs can be made modular using functions, which encapsulate reusable code blocks that perform specific tasks. This improves code readability and maintainability.

The basic syntax for a function:

<returnType> <functionName> (<FormalParameterList>) {
    <FunctionBody>;
}

You can have:

- Functions without return types and parameters.
- Functions with return types and no parameters.
- Functions with parameters and no return types.
- Functions with both parameters and return types.

Function overloading allows multiple functions to have the same name but different parameters, enabling behavior variations based on input types or number. Default parameters provide default values for function arguments, allowing functions to be called with fewer arguments.

In this section we will consider functions that accept references and parameters and look at how this differs from using return values. We will consider when we need to make parameters or variables immutable before having a quick look at recursive functions.

7.2.1 Pass by Reference

To date, all parameter passing to functions has been by value. This means that a value is sent to the function and if changed in the function, it does not impact the values after the function has successfully executed. Effectively, copies of the values are made and used in the function and when the function returns, these copies are destroyed without impacting the values of the variables used to call the function.

Take the following C++ program, with a `nextBirthday` function, as an example.

```
#include <iostream>

using namespace std;

void nextBirthday(int value) {
    value++;
}

int main() {
    int age;

    age = 18;
    nextBirthday(age);

    cout << "Congratuations on turning " << age << endl;
    return 0;
}
```

Listing 7.1: `nextBirthday` pass by value and no return type

After calling the function `nextBirthday`, the value of `age` that is written to the console has not changed from what it was before the function was called. One way to rectify this is to write the function so that it returns the updated age. This function will also need to pre-increment the parameter `value` to ensure the value returned has been incremented. Refer to Listing 7.2.

```
int nextBirthday(int value) {
    return ++value;
}
```

Listing 7.2: `nextBirthday` pass by value and `int` return type

This solution will require the calling code to be updated to age = nextBirthday(age);, which is confusing to say the least and not the most readable and understandable code.

There are therefore instances where one would want to modify the value of a parameter (or parameters) inside the function and have the change reflect after the successful execution of the function. To facilitate this functionality, the parameters need to be defined to be passing by reference. The function does not make copies of the actual parameters, but uses the actual parameters by referring to where they are stored in memory. The `nextBirthday` function, rather than returning a value, will be updated to defining its parameter as a pass by reference parameter. Refer to Listing 7.3 that implements of the original function (Listing 7.1) to accept the parameter using pass by reference (the & before `value` indicates pass by reference).

```
void nextBirthday(int &value) {
    value++;
}
```

Listing 7.3: `nextBirthday` pass by reference no return type

Now, incrementing value in the function will result in the actual parameter being changed and the change reflected in the calling code.

Pass by reference is useful, especially when a function manipulates more than one parameter value that needs to be accessible to the code that called the function. Remember, a function can only return one value using the return statement.

A function to swop values is given to illustrate the use of pass by reference where more than one value is to be returned to the calling code. This is an adaption of Plan 6.H, with the same name, presented in Study Unit 6.

Plan 7.A: Swop two values

General code structure

```
void swop(int &a, int &b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

Example

```
#include <iostream>

using namespace std;

void swop(int &a, int &b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main() {

    int x, y;

    x = 10;
    y = 20;

    swop(x,y);

    cout << "x=" << x << "and y=" << y << endl;

    return 0;
}
```

Here, the & (reference operator) is used to indicate pass by reference.

When the `swop` function is called with arguments `x` and `y`, the parameters `a` and `b` inside the `swop` function become aliases for `x` and `y` respectively. This means that `a` and `b` do not have their own separate memory locations; instead, they refer to the same memory locations as `x` and `y`. Therefore, any modifications to `a` or `b` directly affect `x` and `y`.

This direct referencing is achieved using the reference operator (`&`) in the function declaration, which ensures that the variables are not copied but rather referenced directly. This is why after the `swop` function executes, the values of `x` and `y` in the main program are exchanged. The memory that is being referred to by `a` and `b` inside the `swop` function is the same memory that holds `x` and `y`.

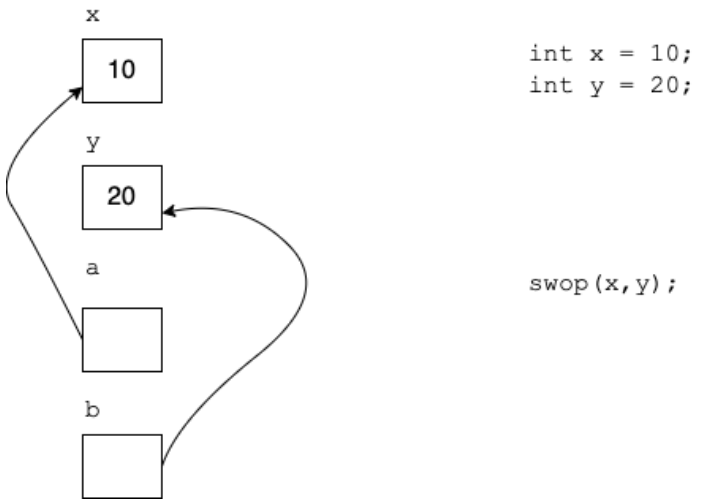


Figure 7.1: Call to the Function `swop` - prior to execution of the body of `swop`

Note, only pass the index of a loop by value in order to ensure it is not accidentally manipulated by the function.

When calling functions that use pass by reference, the actual arguments must be variables that exist in memory, because the reference operator (`&`) in the function's signature requires a memory address to bind to. A reference needs an existing object (or variable) to refer to.

Passing literals (such as `5`, `10.5`, or `'a'`) or expressions (like `x + y` or `z * 2`) directly into a function that expects references will result in a compilation error. This is because these values are not at a specific memory address that the reference can bind to, unlike variables which do have fixed memory locations.

The following would cause a compilation error:

```
#include <iostream>

void increment(int &num) {
    num++;
}

int main() {
    increment(5); // Error: cannot bind non-const lvalue reference to an rvalue
    return 0;
}
```

Listing 7.4: Incorrect Call to Pass by Reference Function

To correctly call the `increment` function, you must provide a variable:

```
#include <iostream>

void increment(int &num) {
    num++;
}

int main() {
    int value = 5;
    increment(value); // Correct usage
    std::cout << "Incremented value: " << value << std::endl;
    return 0;
}
```

Listing 7.5: Correct Call to Pass by Reference Function

`value` is a variable with a memory location, so it can be successfully passed by reference to the `increment` function.

7.2.2 Constness

Constness refers to the immutability of variables and parameters. This means they cannot change. We have used the keyword `const` to define constant variables such as **const float** PI = 3.1415; in Study Unit 3. Later in this study unit you will see how `const` is used to ensure that memory addresses and values being pointed to in memory cannot be changed. In this section we will focus on defining the constness of function parameters.

Consider the following example.

```
int square(const int x) {
    return x * x;
}
```

Listing 7.6: `const` Function Parameters

In this example ***x** is an int that is constant* (read the parameter definition from right to left). This means, within the function the value of `x` cannot be changed. When pass-by-value without the `const` keyword, we could change the value of `x` inside the function without changing the value of the actual parameter with which the function was called. For example, **int** a = 10; square(a);. Making the value a constant means that the value cannot be changed within the function. Therefore, x +=2; within the function body would not compile.

If the parameter in Listing 7.6 was defined as **const int**& x, then the value of `x` cannot be changed in the body of the function. Effectively making it equivalent to passing the parameter by value.

7.2.3 Simple recursion

Recursive functions are functions that call themselves until a condition is met. Each time the function calls itself, the state of the current iteration is placed on the runtime stack.

When the condition is met, the previous state is removed from the runtime stack and that iteration of the function continues executing.

Consider the power function, a^b. The function takes two parameters, the first is the base (a), and the second the exponent (b). An iterative version, that is a version were there is no call to the function itself, of the function is given by:

```
int f(const int a, const int b) {
    int pow = 1;
    for (int i = 0; i < b; i++) {
        pow *= a;
    }
    return pow;
}
```

Listing 7.7: Iterative Version of the Power(f) Function

All recursive implementations of functions require a definition for the recursive call and the conditions under which this call may be made; and a stop condition - the conditions required for the function to begin removing the previous recursive call from the runtime stack. This is sometimes referred to as unwinding.

For the power function, we turn to Mathematics for the formal definition of a recursive power function. The formula for the recursive definition for the power function is given by:

$$f(a,b) = \begin{cases} 1 & if\ b = 0 \\ a * f(a, b-1) & if\ b > 0 \end{cases}$$

The function will call the second statement when $b > 0$, which in turn will call the function with parameters a and $b - 1$. When b is 0, the function no longer calls recursively as it has reached its stop condition and begins to unwind.

To illustrate the recursive calls and the unwinding of the recursive calls, consider the recursive function calls for the function when called with a = 2 and b = 3 (that is $f(2,3)$).

```
2 * f(2,2)
2 * (2 * f(2,1))
2 * (2 * (2 * f(2,0)))
2 * (2 * (2 * (1)))
```

The function then begins to unwind:

```
2 * (2 * (2 * (1)))
2 * (2 * (2))
2 * (4)
8
```

Resulting in the answer of 8. The code for the function is given by:

```
int f(const int a, const int b) {
    if (0 == b) {
        return 1;
    }
    return a * f(a,b - 1);
}
```

Question 7.1
Write a function to determine the the factorial of a number. For example,
$8! = factorial(8) = 8 * factorial(7)$.
Ask yourself, what is the recursive call and what is the stop condition? Try write the recursive function definition mathematically before you attempt to code it.

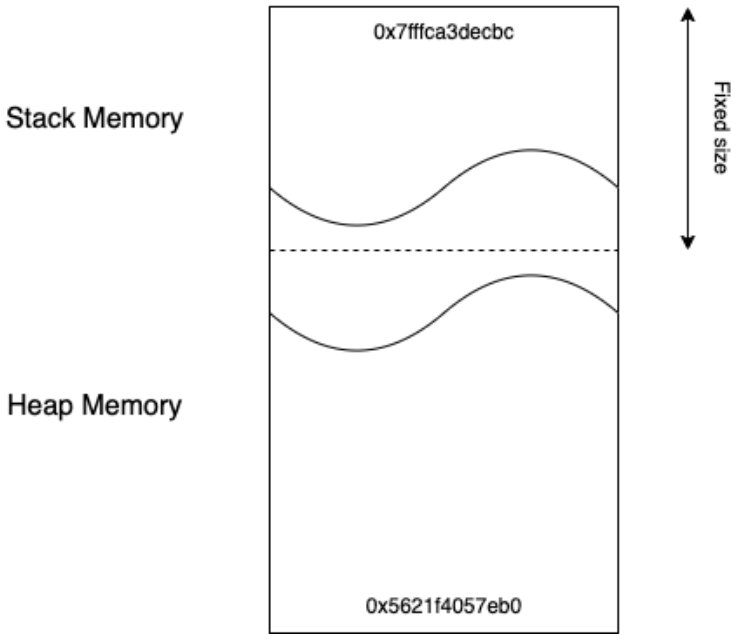
7.3 Allocating and Deallocating Memory

7.3.1 Stack and Heap Memory

Stack memory stores local variables and the state of a function when it is called. It is defined in the higher primary memory address space. Each allocation is done in memory below the previous allocation in the address space. The allocation and deallocation of stack memory is automatic. The lifetime and size of items stored in stack memory is predictable. Memory space allocated for use as stack memory is fixed in size.

Heap memory, the programmer needs to allocate and deallocate. Allocation of heap memory is done using the `new` construct and deallocation with the `delete` construct defined in C++. For every call to `new` in the program, there must be a corresponding call to `delete`. Heap memory is in the lower primary memory address space. Inappropriate management of heap memory can lead to run-time errors such as segmentation faults. Heap memory is allocated dynamically, the size of the item to be allocated and and the lifetime are not known at compile-time, but are determined at run-time.

As the program allocates stack and heap memory, the stack and heap addresses approach each other in the available memory space. When they meet, there is no more memory available to allocate and the if no deallocations have occurred to free memory, the program crashes due to being out of memory.



We will consider the allocation of variables and their values both on the stack and on the heap. When allocation is done on the stack, both the value and the variable as placed on the stack. The memory address where the value of the variable is placed is called the reference. In contrast, when heap memory is allocated, a pointer is placed on the stack. This pointer points to a memory address in heap memory where the value that is associated with the pointer resides.

7.3.2 References

A reference is another name for an existing variable. It links to the memory address where the value represented by the variable resides. Consider the following example:

```
int x = 10;
int &ref = x;
```

In this code, `x` is a variable with value 10, stored on the stack. `ref` is a reference variable that refers to the same memory location as `x` and also stored on the stack. Changing the value using either `x` or `ref`, changes the value stored at the memory address `&x`.

Remember, that when the variable goes out of scope, the memory on the stack is deallocated. This means that if a reference is returned by a function and the function terminates, the memory is no longer allocated to the particular variable.

What happens when we pass a value of a variable on the stack by reference to a function? Consider the implementation of the `nextBirthday` function given below.

```
int& nextBirthday(int &value) {
    value++;
    return value;
}
```

Listing 7.8: `nextBirthday` pass by reference and a reference to an int return type

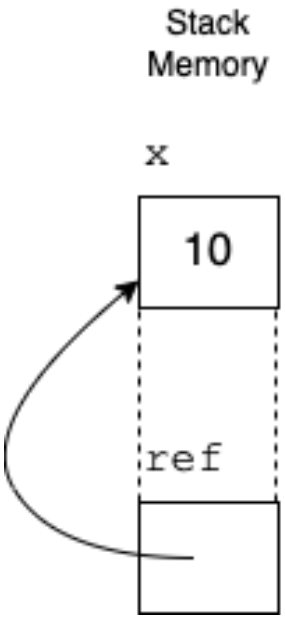


Figure 7.2: References on the Stack

You may assume that the function is called as follows with age defined as in Listing 7.1:

int newage = nextBirthday(age);

Figure 7.3 provides a graphical representation of how stack memory is managed for the call to the function `nextBirthday` given in Listing 7.8. When the function returns, it is effectively saying that `newage = &value` which copies the value in `age` into `newage`. `age` and `newage` are independent of each other.

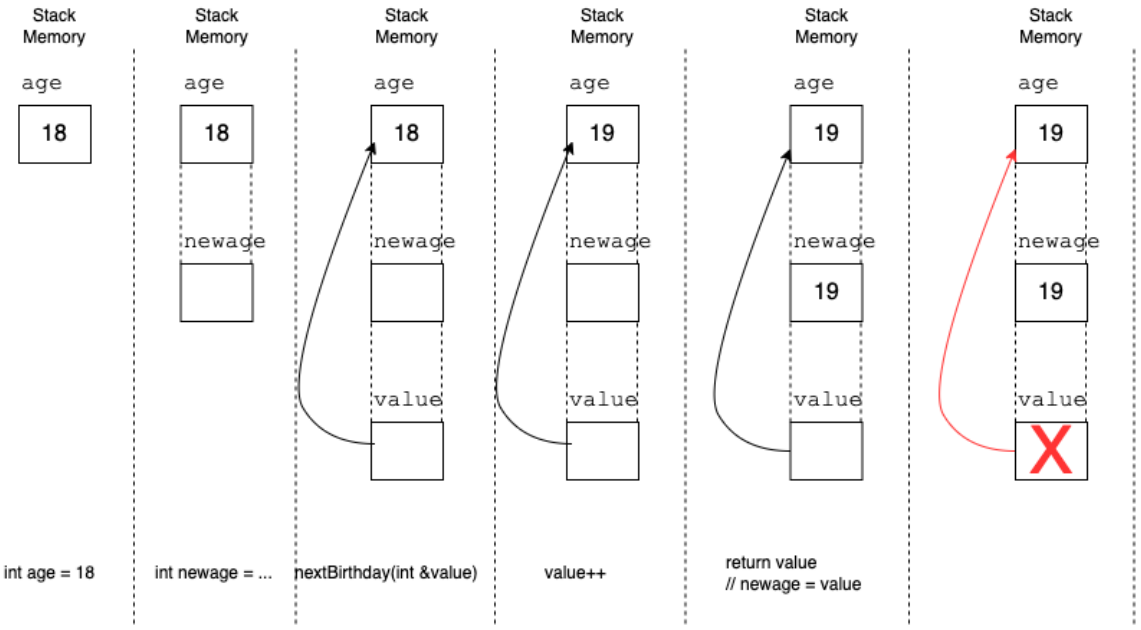


Figure 7.3: Calling the function `int& nextBirthday(int &value)`

7.3.3 Pointers

A pointer is a variable that resides in stack memory that holds a memory address. This memory address points to a location typically in heap memory that holds a value. For example, **int** ∗p; defines a variable `p` in stack memory, that will hold a memory address on the heap, once memory has been allocated. To allocate memory, the `new` operator is used. In our example, `p` is defined as a pointer to a value of type `int`. Therefore, to allocate memory to which `p` is pointing, the following statement is required: p = **new int**;. Once heap memory has been allocated, values can be placed in the reserved memory. To do this the statement ∗p = 10; can be used to allocate the integer `10` to the memory. Applying the `*` operator to a pointer variable is referred to as dereferencing. A value, if known when the memory is allocated, can be assigned when the `new` operator is called, that is p = **new int** (10);. The following figure illustrates the memory after `new` has been called.

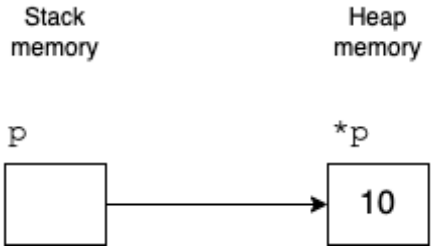


Figure 7.4: Pointers on the Heap

The series of C++ statements, given in Figure 7.5, illustrate how memory is allocated, used and deallocated.

All memory that has been allocated with a `new` by the programmer must be deallocated using the `delete` operator. This will release the memory back to the operating system and free it up for either the same program to reuse or another program to use. Once the memory has been freed, it is good programming practice to "null" the pointer variable. This enables for comparison against a known value. To "null" the pointer variable, the values of NULL, 0 or nullptr can be assigned to it. If no memory is being allocated, it is best to assign `NULL` (or `0`) to `p`.

7.3.4 Working with references and pointers

Consider the `nextBirthday` function that is passed a pointer as a parameter.

```
void nextBirthday(int *value) {
    (*value)++;
}
```

and is called as follows:

```
nextBirthday(&age);
```

Since the `age` variable is allocated on the stack we're not dealing with any heap allocations for this particular case, so all memory will be on the stack.

- The `age` variable is stored on the stack at some memory address.

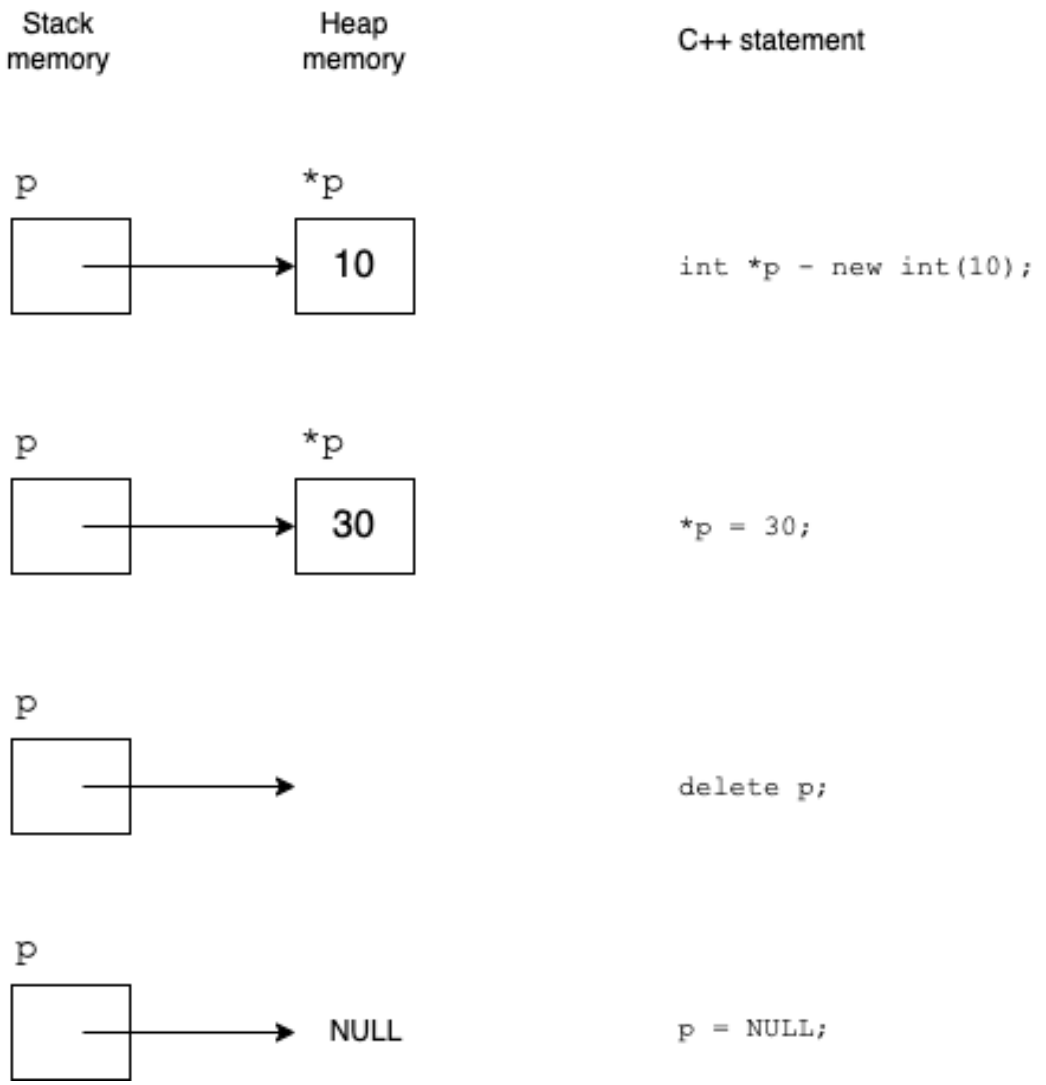


Figure 7.5: Allocating memory on the heap

- When `nextBirthday(&age)` is called, the address of age is passed to the function. This means the pointer inside nextBirthday points to the age variable's memory location.
- (*value)++ in the function dereferences the pointer (accesses the memory location the pointer refers to) and increments the integer stored there.

We will go into this in more depth when we look at arrays and structs.

When a function allocates memory on the heap and returns the pointer to the memory on the heap, the onus is on the code that has called the function to clean the heap memory and no longer on the function that called `new`.

Question 7.2
How does constness impact references and pointers when applied to functions? Experiment with a few functions to see what the outcome is. Here is a sample of possible functions to experiment with.

- **void** printConstRef(**const int**& num) {

```
    // num++; // Uncommenting this line will cause a compile error
                // because 'num' is const.
    cout << "The number is: " << num << endl;
}
```

- ```
  void printConstPointer(const int* ptr) {
      // *ptr = 10; // Uncommenting this line will cause a compile
                    // error because '*ptr' is const.
      cout << "Pointer points to: " << *ptr << endl;
  }
  ```

- ```
  const int& getConstRef() {
      static int value = 10;
      return value;
  }
  ```

- ```
  void printConstPointerToPointer(const int** ptrptr) {
      // **ptrptr = 10; // Uncommenting this line will cause a
                        // compile error because '**ptrptr' is const.
      *ptrptr = NULL; // Legal since the int ** is const,
                      //not the int *
      cout << "Pointer to pointer points to: " << **ptrptr << endl;
  }
  ```

- ```
  void increment(int* const ptr) {
      (*ptr)++;
      cout << "Incremented value: " << *ptr << endl;
  }
  ```

7.3.5 Errors and exceptions when working with pointers

A segmentation fault (or segfault) is a common error in C++ that occurs when a program tries to access a memory location that it doesn't have permission to access. This typically happens when a program attempts to read or write to a memory location outside the boundaries of the memory that has been allocated for its use. This can occur due to various reasons, such as dereferencing NULL or uninitialised pointers, accessing memory that has been freed (dangling pointers), or overstepping the bounds of allocated memory.

When a segfault occurs, the operating system usually terminates the program and generates a *core dump*. This dump contains a snapshot of the program's memory at the time of the fault, which can be analysed to determine what caused the error.

Scenarios that often lead to segmentation faults includes:

Dereferencing NULL Pointers : (Null pointer exceptions) Accessing memory through a pointer that has not been initialised or is set to NULL leads to undefined behaviour and can cause a segmentation fault.

```
#include <iostream>

int main() {
    int* ptr = NULL; // Pointer initialized to NULL
    std::cout << *ptr; // Dereferencing NULL pointer causes segfault
    //Instead perform a check
    if(ptr != NULL){
        std::cout << *ptr;
    }
    return 0;
}
```

Dereferencing Freed Memory: Accessing memory through a pointer to memory that has been freed (deleted) can lead to a segmentation fault. This is because the memory might no longer be allocated to the program.

```
#include <iostream>
#include <cstdlib>

int main() {
    int* p = new int(10);
    delete p; // Memory freed
    *p = 5; // Accessing freed memory causes segfault
    std::cout << *p;
    return 0;
}
```

Buffer Overflow: Writing data beyond the allocated buffer can overwrite adjacent memory, leading to unpredictable behaviour and potentially a segmentation fault.

Stack Overflow: Using too much stack memory, typically through deep or infinite recursion, can lead to a stack overflow which is a type of segmentation fault. When this happens, make sure that your base case for your recursion is correct.

```
#include <iostream>
using namespace std;

void recurse() {
    int a = 0;
    cout<<"Recurse"<<endl;
    recurse(); // No "end" to recursion
}

int main() {
    recurse(); // Causes stack overflow
    return 0;
}
```

Improper Memory Management: Incorrect management of dynamic memory, such as double freeing (deleting the same thing twice) or mismatched use of new/delete, can lead to segmentation faults.

```
int main(){
    int* ptr = new int;
    delete ptr;
    delete ptr; // Double free leads to undefined behavior
                // potentially a segfault

}
```

Some tips to avoid/fix segmentation faults:

- **Initialise Pointers:** Always initialise pointers. Uninitialised pointers can point to random memory locations, causing segfaults when dereferenced.
- **Validate Pointers**: Before dereferencing pointers, check if they are NULL or have been assigned valid memory addresses (use if statements).
- **Manage Memory Carefully**: Ensure that memory operations such as allocation and deallocation are correctly matched and that no memory is accessed after it has been freed.
- **Boundary Checks:** Always perform boundary checks when accessing arrays or buffers to prevent out-of-bounds access

7.4 Manipulating Strings with Pointers

Strings can be manipulated using pointers, which provide a direct way to access and modify individual characters. A string in C++ can be treated as an array of characters, i.e. a number of characters next to each other in memory. With pointers, you have the ability to iterate over these characters for various manipulations.

C++ differentiates between small strings and strings that are not considered small. A small string is stored on the stack, while all other strings are stored on the heap. For the purposes of this study unit, we will assume that all strings are stored on the heap. A variable is therefore stored on the stack that points to the memory address that the first character of string is store in. Enough memory is allocated to store the number of characters that make up the specific string to be stored, plus the string termination character `'\0'`. The string `"Hello World!"`, will therefore take 12 characters (so `12 * sizeof(char)` bytes) to store in memory.

To access the first character of a string that is defined by `string myString = "Hello World";`, you can treat the string as an array (details of arrays will be discussed in Study Unit 8) and use indexing. The statement `myString[0]` is the first character in the string, whereas `&myString` is the reference to where the first character of the string is allocated in memory.

The termination character is another really important character when it comes to manipulating strings. As strings are variable in length, it indicates where the string ends in memory.

Question 7.3
What will `&myString[0]` result in when written out to the console?

7.4.1 Pointers to Strings

Consider the definition to create a pointer to a string, given by `string* strPtr = new string("Hello, world!");`. Here the actual string is stored on the heap and `*strPtr` is used to access the string. `*strPtr)[0]` represents the first character of the stirng.

Figure 7.6 illustrates how the strings are managed in memory and how they can be manipulated for the two string definition statements of:

string myString = "Hello World!";
string *strPtr = **new** string("Hello, world!");

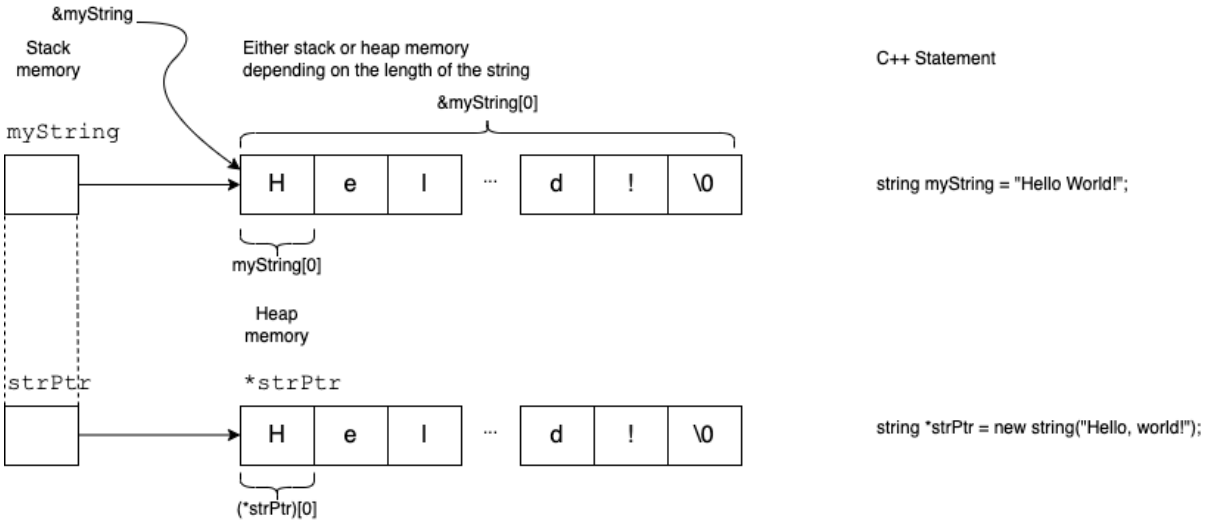


Figure 7.6: Allocating strings

Never increment or decrement the pointer used to create the string. Doing this will result in you loosing your access to the beginning of the string. Always assign the beginning of the string to a different pointer variable, specifically a variable of `char*`.

7.4.2 Iterating Over Strings with Pointers

Iterating over a string with an index is trivial. For `myString`, you can do so with the following while-loop.

```
string myString = "Hello World!";
int i = 0;
while (myString[i] != '\0') {
    cout << myString[i];
    i++;
}
cout << endl;
```

Question 7.4
Write the corresponding while loop to iterate over the `strPtr` as previously defined using an index.

Rather than iterating over the string with an index, we can use pointer arithmetic. This will require a variable of `char *` to be assigned to the first character of the string. At the end of each iteration of the loop, the pointer is incremented by the size of a single character. Below is the code to iterate over the string represented by `myString` is given on the left and `strPtr` is on the right.

```
char *iPtr = &(myString[0]);
while (*iPtr != '\0') {
    cout << *iPtr;
    iPtr++;
}
cout << endl;
```

```
char* jPtr = &((*strPtr)[0]);
while (*jPtr != '\0') {
    cout << *jPtr;
    jPtr++;
}
cout << endl;
```

The next sections will provides exampled of iterating over a string to print the string, determine the length of the string and to revers the string respectively.

7.4.2.1 Print the string

To write a function to print the string using pointers, the function will need to pass the string by reference to make provision for both the string variables. The implementation of the function is based on the two implementations previously given. The

```
void printString(string &str) {
  char *current = &str[0];
  while (*current != '\0') {
      cout << *current;
      current++;
  }
  cout << endl;
}
```

As the function accepts a reference to the string as a parameter, for myString the function is called directly using the variable as is. That is: printString(myString);. For `strPtr`, the dereferenced version of the variable is used, that is printString(*strPtr);.

7.4.2.2 Length of a string

The function to determine the length of a string is similar to printing a string. The function accepts a reference to a string as a parameter and returns an integer representing the number of characters that string comprises of excluding the string termination character. The listing below provides the implementation of a `stringLength` function that makes use of pointer arithmetic.

```
int stringLength(string &str) {
    int len = 0;
```

```
    char *current;
    current = &str[0];
    while (*current != '\0') {
        len++;
        current++;
    }
    return len;
}
```

7.4.2.3 Reversing a string

To reverse a string using pointers, you use two pointers: one pointing at the start of the string and the other at the end. You then swop the characters at these pointers, moving them towards the center until they meet or cross each other.

The following function will use pointers to reverse an object of type `string`.

```
void reverseString(string &str) {
    if (stringLength(str) != 0) {
        char* end = &str[0]; // Pointer to the end of the string
        // Move the 'end' pointer to the last character (\0) of the string
        while (*end != '\0') {
            end++;
        }
        end--; // Set 'end' to the last character (not the terminator)

        // 'start' pointer to the beginning of the string
        char* start = &str[0];

        // Swop the characters until 'start' and 'end' pointers meet or cross
        while (start < end) {
            swop(*start,*end);
            start++;
            end--;
        }
    }
}
```

Listing 7.9: Reversing a `string` object

The function is called as follows for `myString` and `strPtr` respectively.

```
    reverseString(myString);
        cout << "myString reversed = " << myString << endl;

        reverseString(*strPtr);
        cout << "strPtr reversed = " << *strPtr << endl;
```

Question 7.5
Write the `swop` function. The definition of the function is given by: **void** swop(**char** &a, **char** &b);

The function to reverse an object of type `string`, given in Listing 7.9 does not work for strings that are defined as c-strings. That is an array of characters defined in the following manners:

```
char str[] = "Hello, world!"; // As an array of characters

char *sPtr = new char[30]; // As a pointer to an array of characters
strcpy(sPtr,"Hello World"); // ensures the '\0' is added to the array of chars
```

The `cstring` library needs to be included in order to use `strcpy`.

We can rewrite the `reverseString` function in Listing 7.9 to make use of c-strings to cater for both string defined using the C++ string class as well as strings that are written in the c-style. These c-style strings are equivalent to a one-dimensional array of characters which will be discussed in detail in Study Unit 8. Listing 7.10

```
void reverseString(char *str) {
    if (str) {
        char* end = str; // Pointer to the end of the string
        // Move the 'end' pointer to the last character (\0) of the string
        while (*end != '\0') {
            end++;
        }
        end--; // Set 'end' to the last character (not the string terminator)

        // 'start' pointer to the beginning of the string
        char* start = str;

        // Swop the characters until 'start' and 'end' pointers meet or cross
        while (start < end) {
            swop(*start,*end);
            start++;
            end--;
        }
    }
}
```

Listing 7.10: Reversing strings - `string` class and c-string definitions

Below are examples of how the function can be called based on how the string is defined.

```
string myString = ("Hello World");
reverseString(&myString[0]);
cout << "myString reversed = " << myString << endl;

string *strPtr = new string("Hello, world!");
reverseString(&(*strPtr)[0]);
cout << "strPtr reversed = " << *strPtr << endl;

char str[] = "Hello, world!";
reverseString(str);
```

```
cout << "str reversed = " << str << endl;

char *sPtr = new char[30];
strcpy(sPtr,"Hello World");
reverseString(sPtr);
cout << "sPtr reversed = " << sPtr << endl;
```

Caution! Using pointers for string manipulation requires careful management of memory bounds to avoid common errors such as segmentation faults or memory leaks. Always make sure that your pointer operations remain within the valid range of the string and take special care not to dereference NULL pointers.

7.5 Document Change Log

Date	Change
09 May 2024	Initial preparation and presentation of the document. Preliminary version 0.1